BiblioTech

Data: 02/01/2026

Giacomo Cialoni
0311649

Ingegneria del Software e progettazione web

1. SOFTWARE REQUIREMENT SPECIFICATION

1.1. INTRODUCTION

1.1.1. AIM OF THE DOCUMENT

The purpose of this document is to present the fundamental and specific requirements of the BiblioTech Management System in a clear and objective manner.

This document represents the Software Requirement Specification (SRS) of the system and provides an overview of both its internal and external aspects.

The description is supported by diagrams that illustrate the system's structure, behaviour, and interactions among its components.

1.1.2. OVERVIEW OF THE DEFINED SYSTEM

The BiblioTech Management System is an application designed to efficiently manage the activities of a library.

The system allows users to browse and search for books, check their availability, place reservations for purchases or loans, manage their personal profile, and view news and informational posts published by the system administrators.

Administrators are provided with functionalities to manage the book catalog, accept or reject reservations, record sales and loans, and create informational posts to keep users updated about system-related news.

The system also includes advanced search and filtering features, enabling users and admins to quickly locate the desired results.

Furthermore, it supports the creation and management of new resources and categories, making the system flexible and easily extensible.

1.1.3 HARDWARE AND SOFTWARE REQUIREMENTS

The basic hardware requirement of the system is a computer capable of running Java Archive (JAR) files.

Additional disk space is required to store user-related data when the system is used in offline mode.

Regarding software requirements, the system requires Oracle JDK SE 21 to be correctly installed in order to execute the JAR file.

For the online version of the system, a server or a local server hosting a Database Management System (DBMS) is required, allowing the system to retrieve and store information through SQL queries.

1.1.4. RELATED SYSTEMS, PROS AND CONS

Several systems related to the BiblioTech Management System are considered below.

KOHA

- Pros: Open-source ILS supporting catalog management, loans, user accounts, OPAC access, and standard bibliographic formats.
- Cons: Configuration and maintenance can be complex and require technical expertise.

EVERGREEN

- Pros: Open-source and scalable system designed for catalog and loan management, suitable for large library networks.

- Cons: User interface is less intuitive and customization requires advanced technical skills.

1.2. USER STORIES

1.2.1. US-1

As a User, I want to search for books in the store, so that I can buy them.

1.2.2. US-2

As a user, I want to reserve a book for purchase, so that when I arrive at the store the book is not unavailable.

1.2.3 US-3

As an Admin, I want to manage book sales, so that the system's catalog is always updated.

1.3 FUCTIONAL REQUIREMENTS

1.3.1. FR-1

The system shall provide user and admin registration and login in order to access restricted functionalities and interact with system data.

1.3.2. FR-2

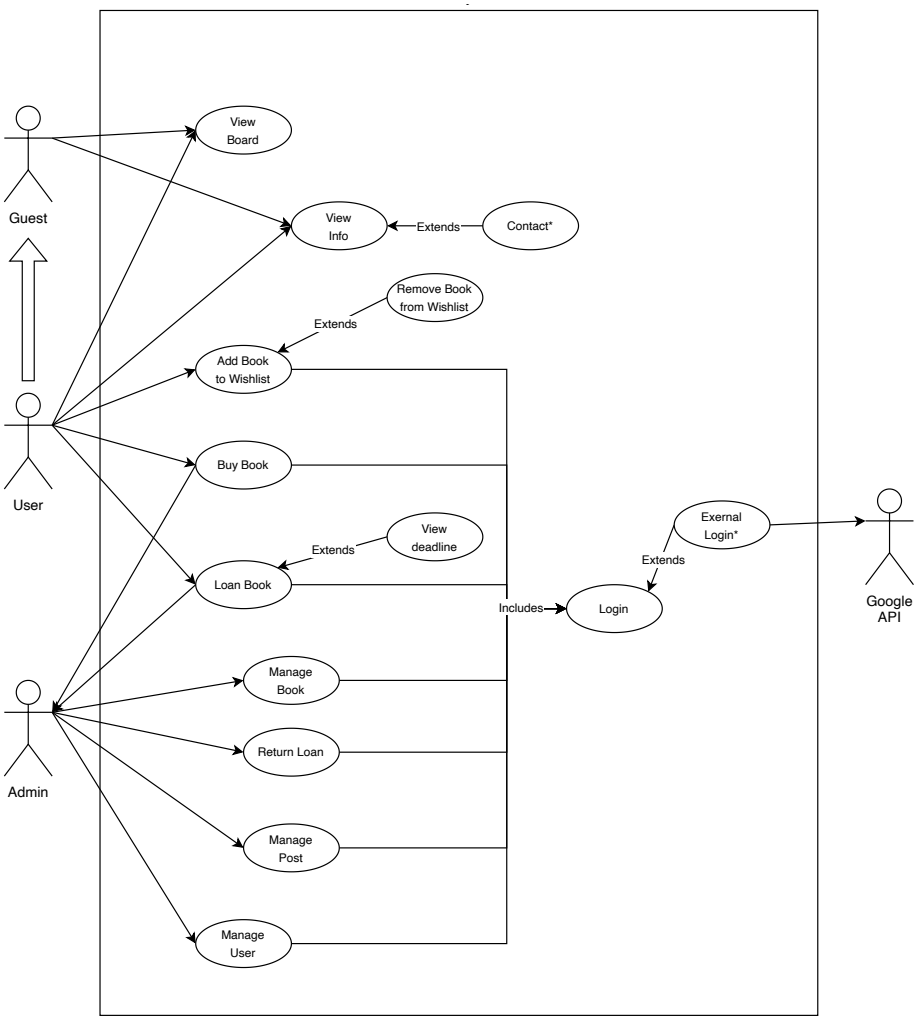
The system shall provide access to a catalog of all books to users.

1.3.3. FR-3

The system shall provide a search function that allows users to find books based on filters so they can find the desired books.

1.4. USE CASES

1.4.1. OVERVIEW DIAGRAM



* funzionalità non implementate nel progetto

1.4.2. INTERNAL STEPS

Loan Book

1. The User request to access the catalog of the books.
2. The System gets books from the Books system.
3. The User selects the desired book.
4. The System gets selected book's details.
5. The User selects to reserve the book to loan.
6. The System checks if the User has correct loan's prerequisites.
7. The System saves the loan and notifies the User of the reservation success.
8. The Admin request to view the reservations.
9. The System gets the reservations that have not yet been managede.
10. The Admin accept the loan.
11. The System save the loan in the system and notifies Admin of the loan success.

Extensions:

6a. User is not logged: System prepares the login schedule.

6b. User has reached the limit of active loans: System notifies the User and terminates the use case.

6c. User has expired loans: System notifies the User to return the expired loan before reserve a new loan and terminates the use case.

7a. Saving loan error: System notifies the User and terminates the use case.

10a. Admin decline the reservation: the loan reservation is deleted from the sysem.

2. STORYBOARDS

When the application is launched, the user accesses the system as a Guest by default. From this state, several primary storyboards are accessible without authentication.

The graphical user interface includes a fixed top navigation menu, composed of clickable buttons that allow access to the main (primary) pages of the system.

When a user navigates to secondary pages by clicking buttons that are not part of the top menu (for example, book details or administrative operations), the top menu is hidden. In these cases, navigation is handled through a “Back” button, which allows the user to return to the previous primary page.

Access to certain storyboards depends on the authentication level:

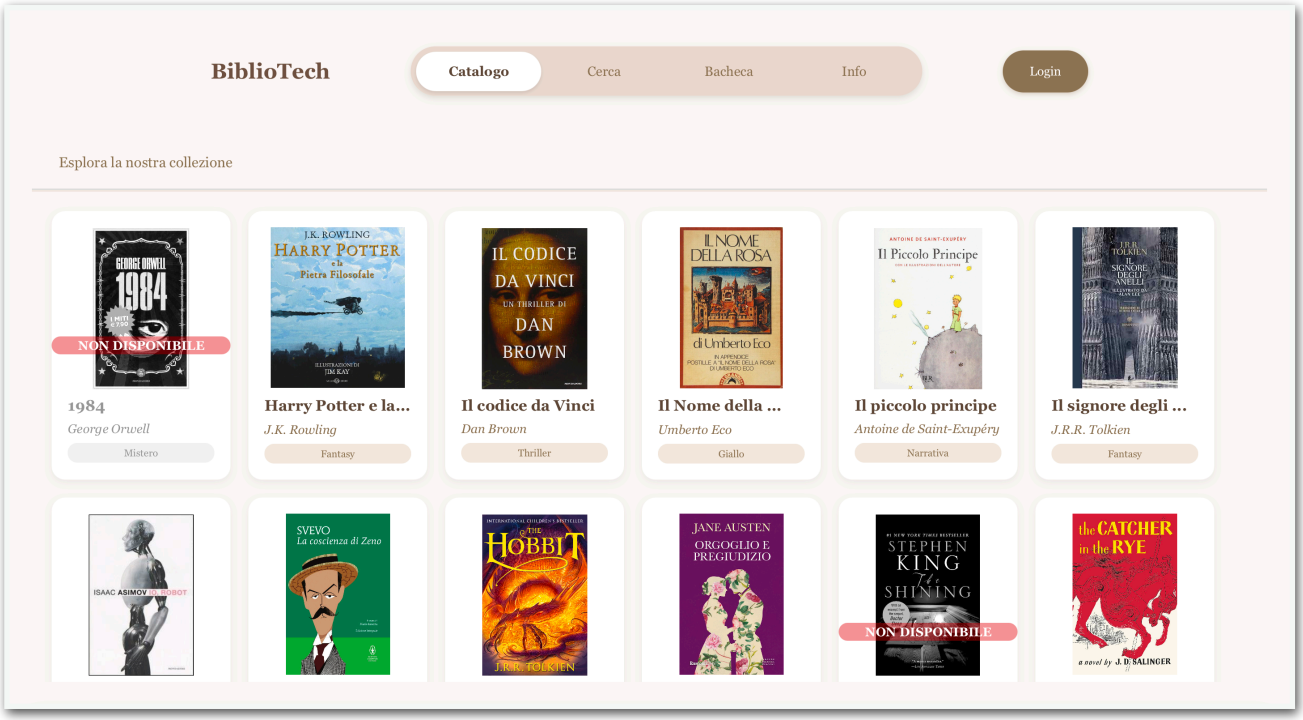
- Guest and User can access public and user-related storyboards.
- Authenticated Users can access user-specific functionalities such as reservations and wishlists.
- Administrators access administrative storyboards after logging in with admin credentials.

The *Catalog* storyboard represents the default screen displayed when entering the system as a Guest.

This storyboard is accessible to both Guests and authenticated Users.

It shows a fixed top navigation menu with clickable buttons that allow users to access the primary features of the system, including a Login button.

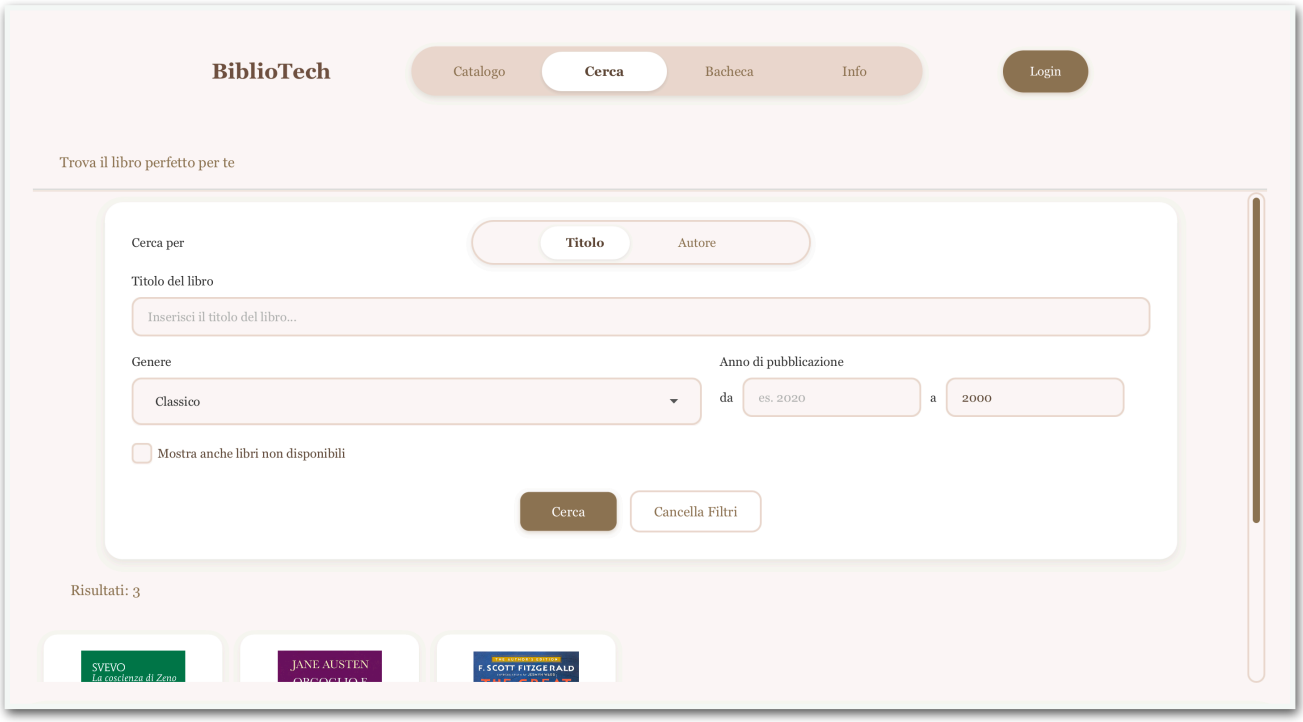
All books in the library are displayed in this screen, enabling users to browse the catalog and select a book to view its details.



The *Search* storyboard is accessible from the top navigation menu and is available to Guests and Users.

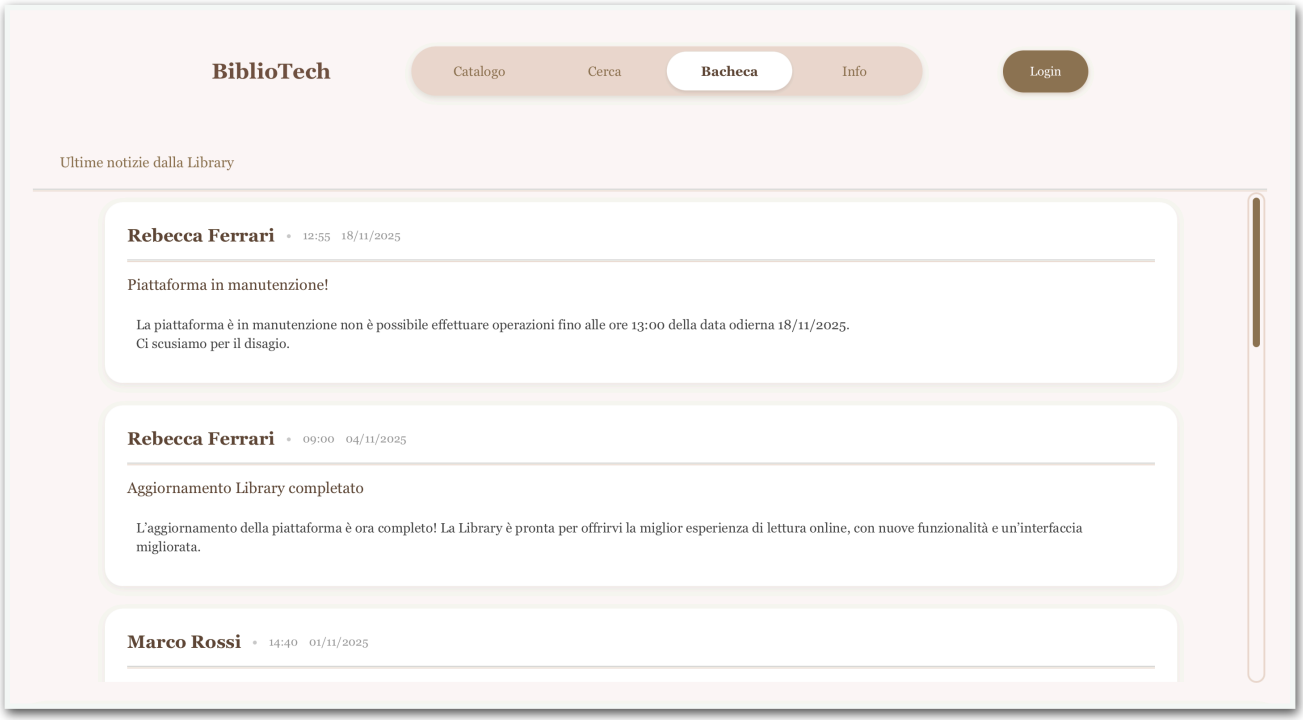
It is designed to help users who already know what they are looking for to quickly find a specific book by using search and filtering options.

This storyboard improves system usability by reducing the time required to locate a desired book.



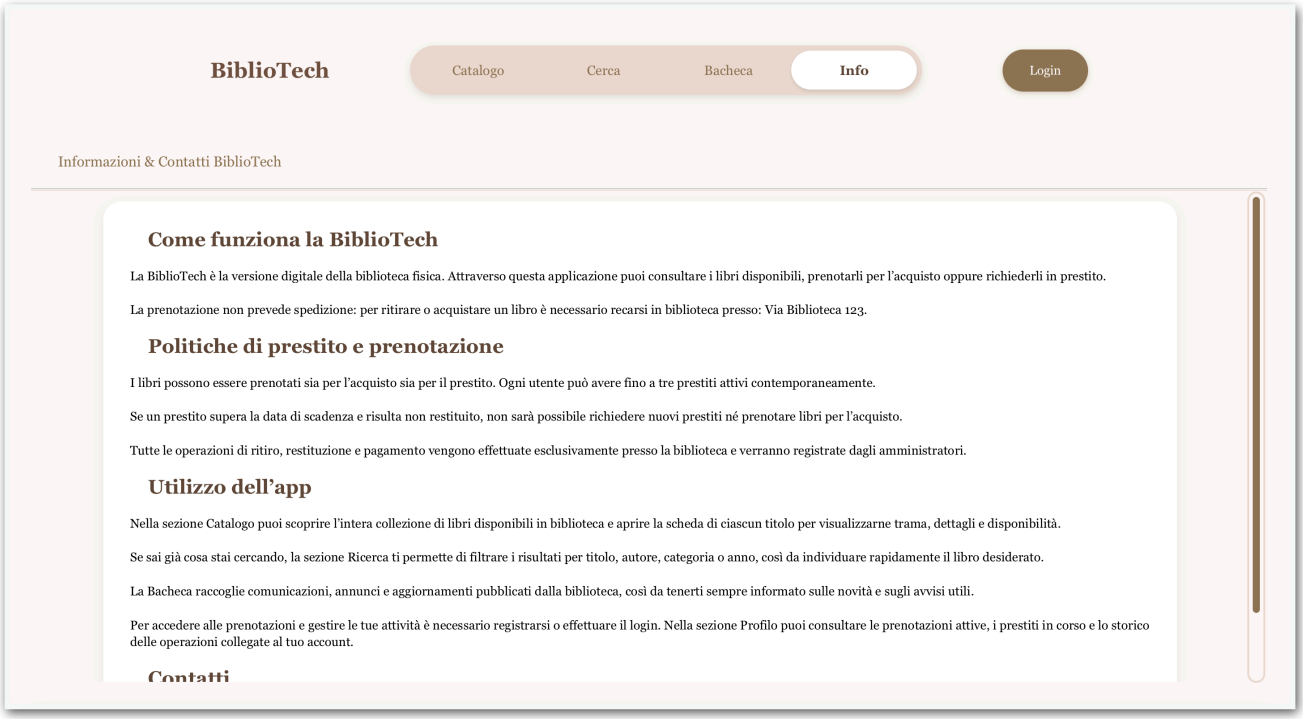
The *Board* storyboard allows Guests and Users to view posts published by BiblioTech administrators.

Posts are displayed in chronological order and provide updates, announcements, and relevant information related to the system.



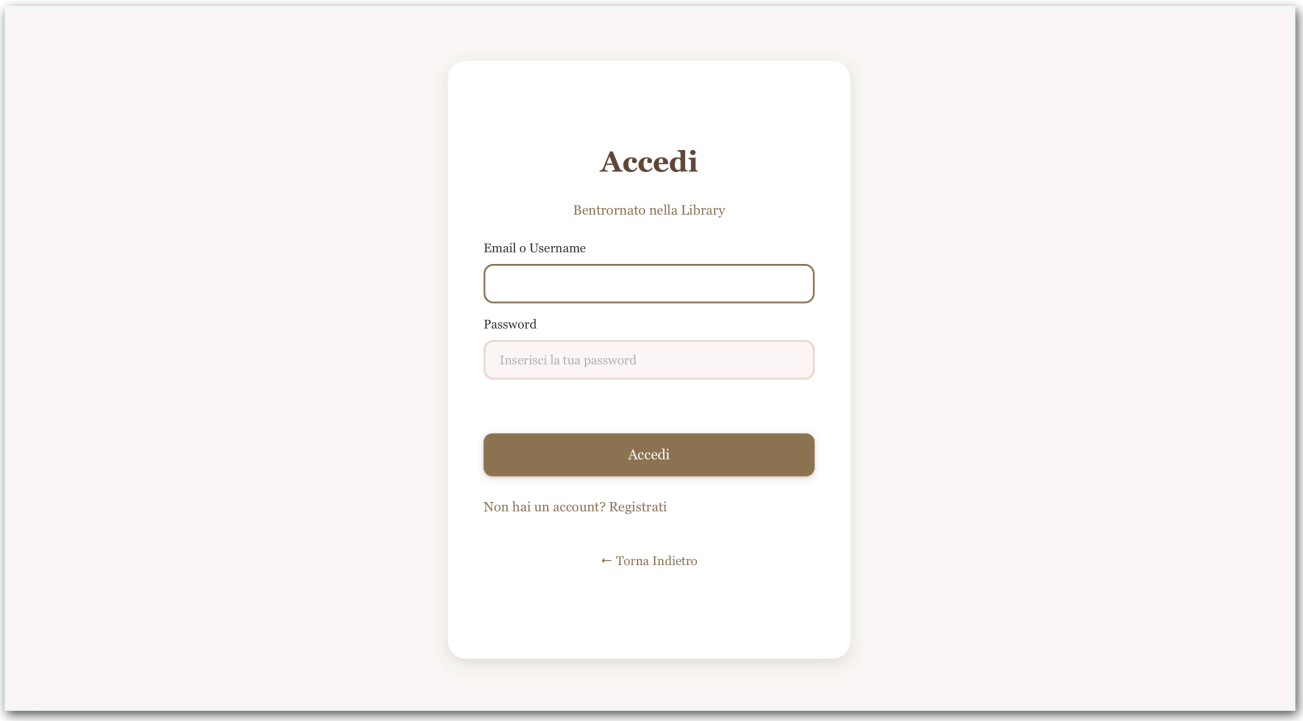
The *Info* storyboard is a static screen accessible to Guests and Users.

It provides information about system policies and general details regarding the BiblioTech platform.

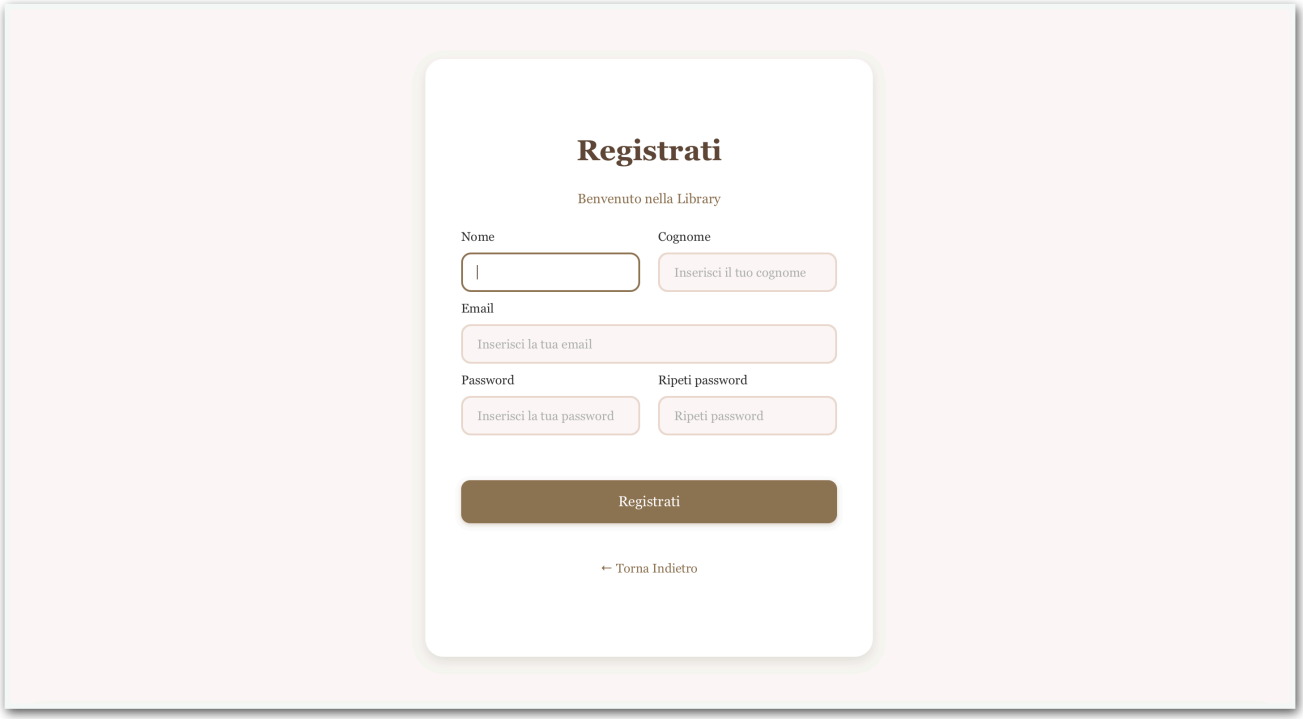


The *Login* storyboard allows Users and Administrators to authenticate themselves in order to access restricted system functionalities.

Authentication is required to perform actions such as reserving books, managing personal profiles, and accessing administrator-specific features.



If a user is not authenticated, they can sign in by filling out the *Sign In* form.

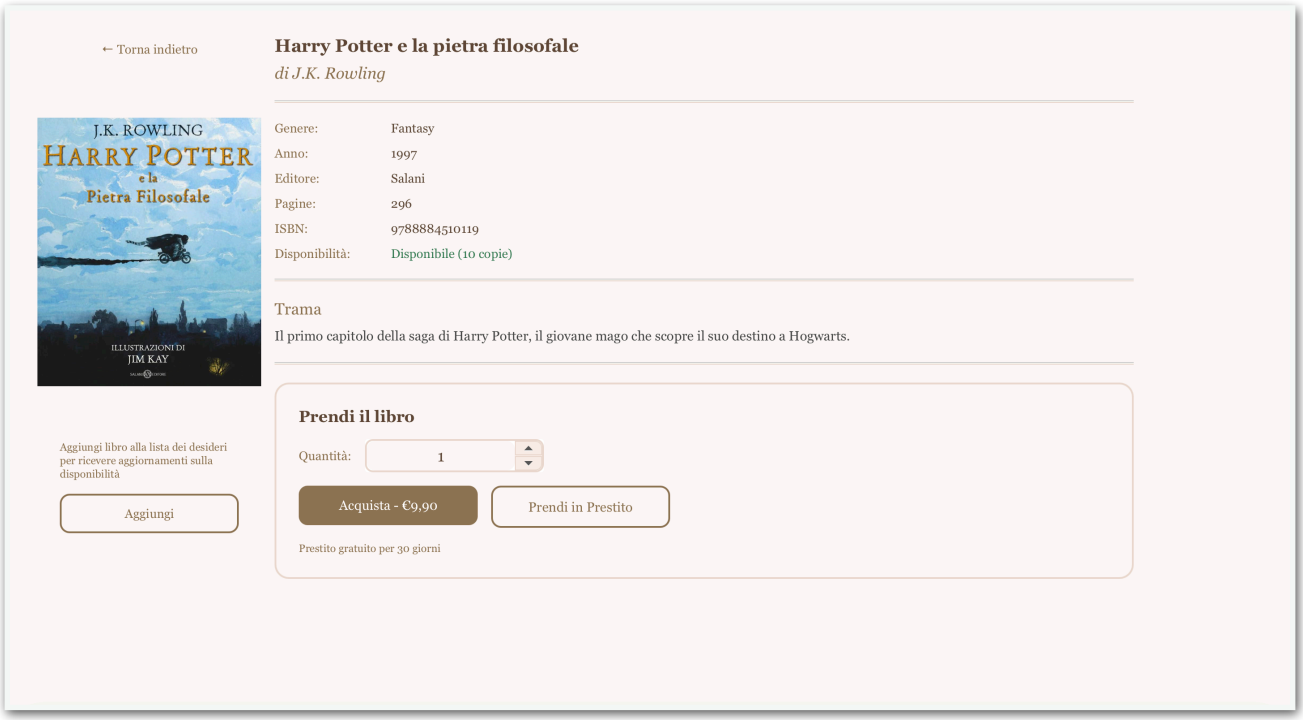


By selecting a book from either the Catalog or Search storyboard, users access the *Book Detail* storyboard, which is considered a secondary page.

This screen displays detailed information about the selected book, including its specifications and availability.

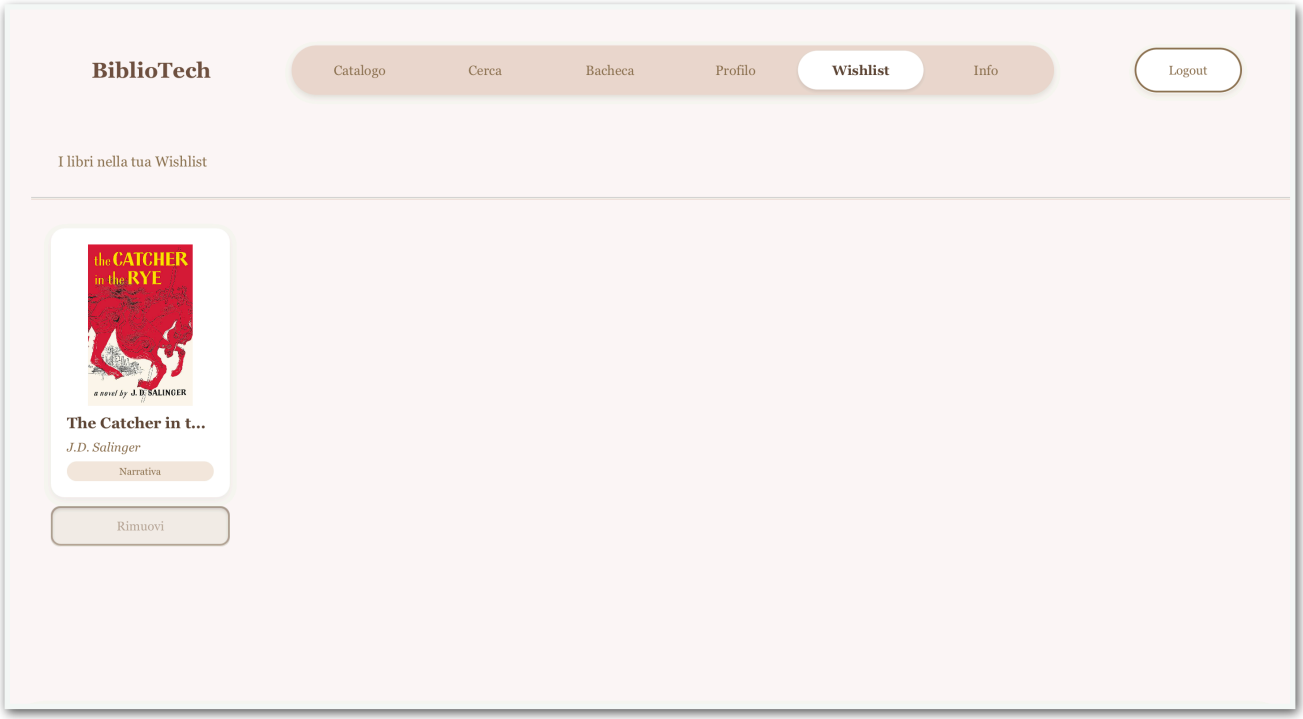
From this storyboard, authenticated Users can reserve the book for purchase or loan and add the book to their wishlist.

If a book added to the wishlist is not currently available, the user will receive an email notification when it becomes available.



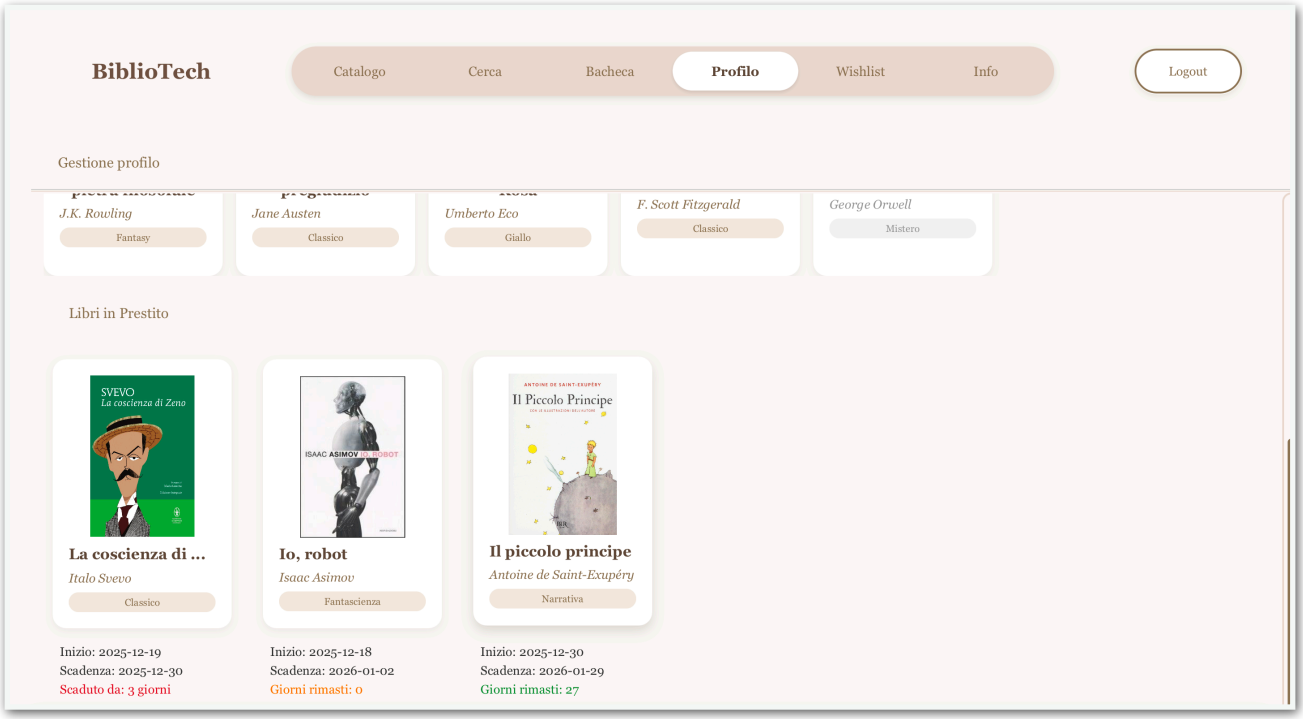
The *Wishlist* storyboard is accessible only to authenticated Users.

It displays all the books that the user has added to their wishlist.



The *Profile* storyboard is accessible only to authenticated Users.

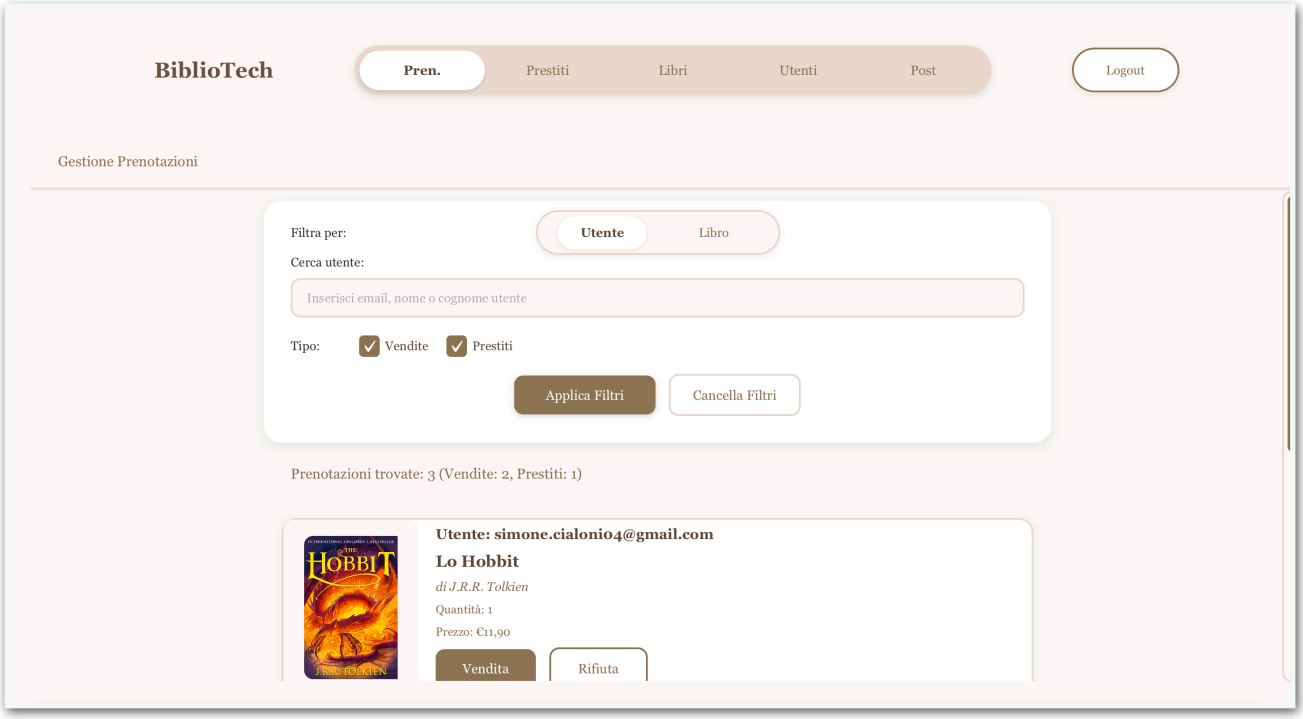
It displays the user's personal information, such as name and email, as well as purchase history and loan status.



When logged in as an Administrator, the *Reservations* storyboard is the default screen displayed after authentication.

This storyboard allows administrators to view and manage reservations made by users that have not yet been accepted or rejected.

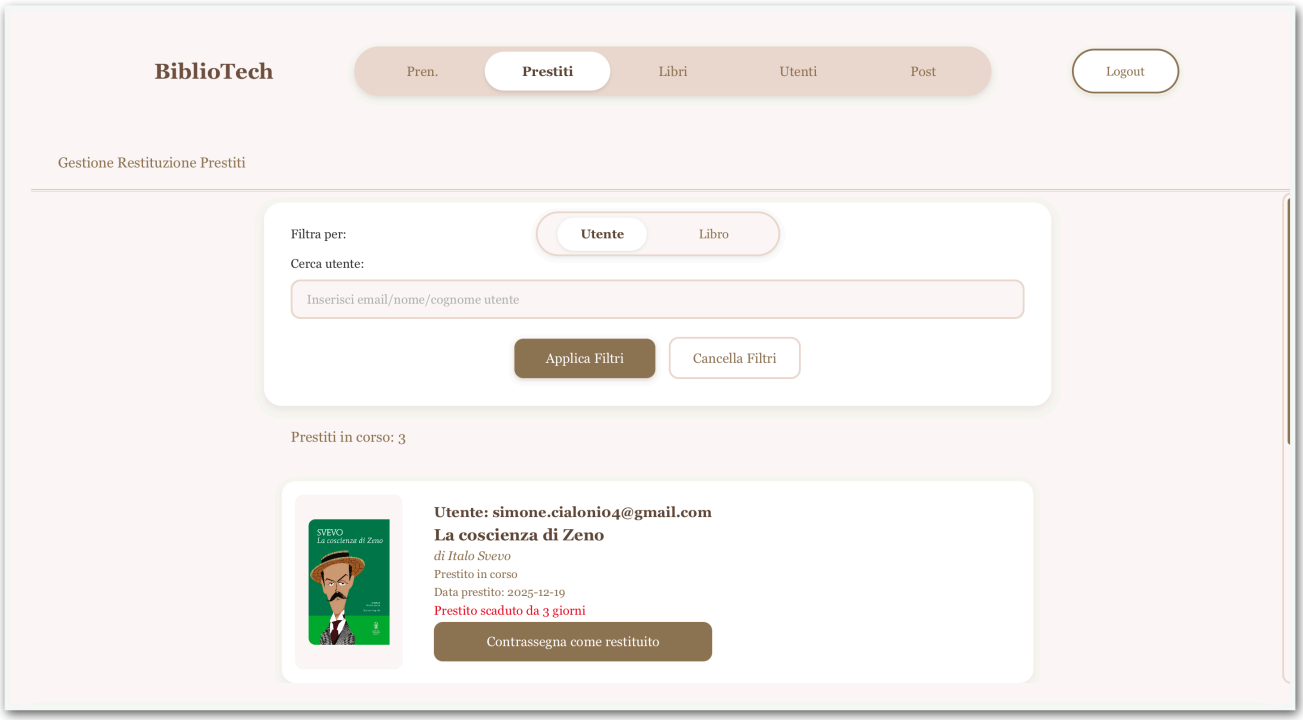
Administrators can scroll through all reservations or filter them by user, and can accept or reject purchase or loan requests.



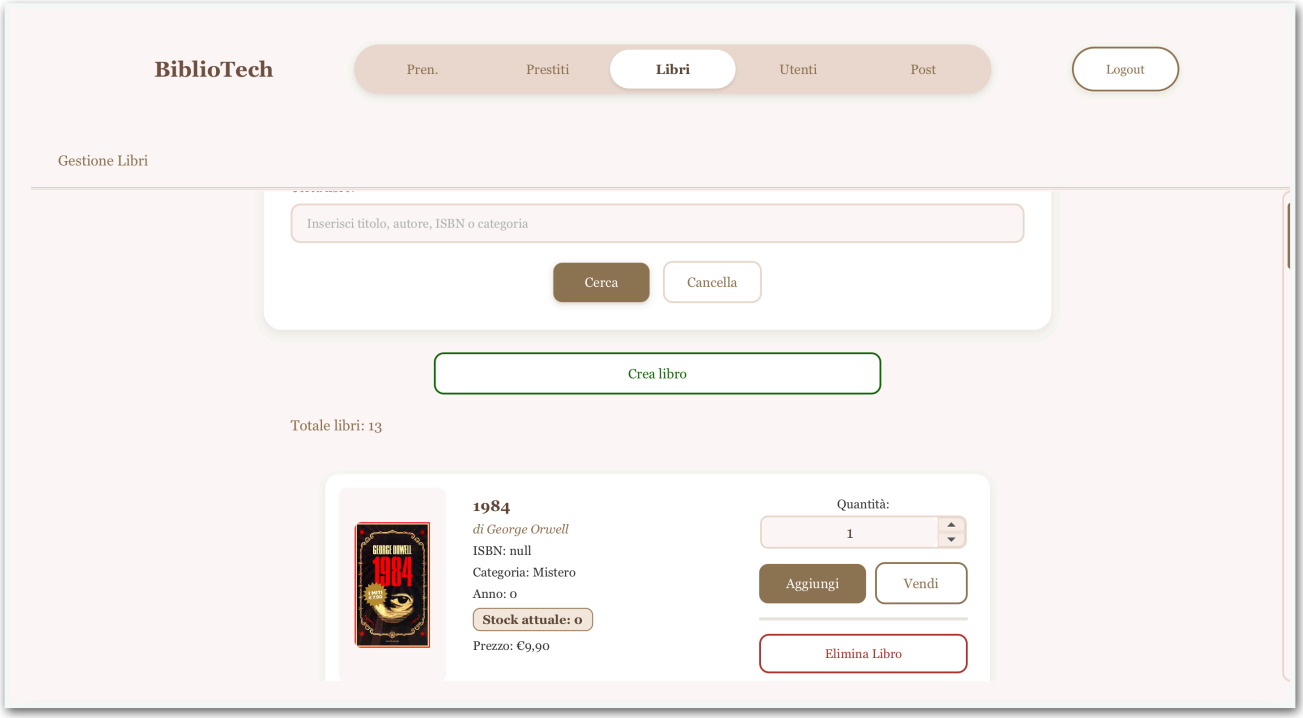
The *Loans* storyboard is accessible only to Administrators.

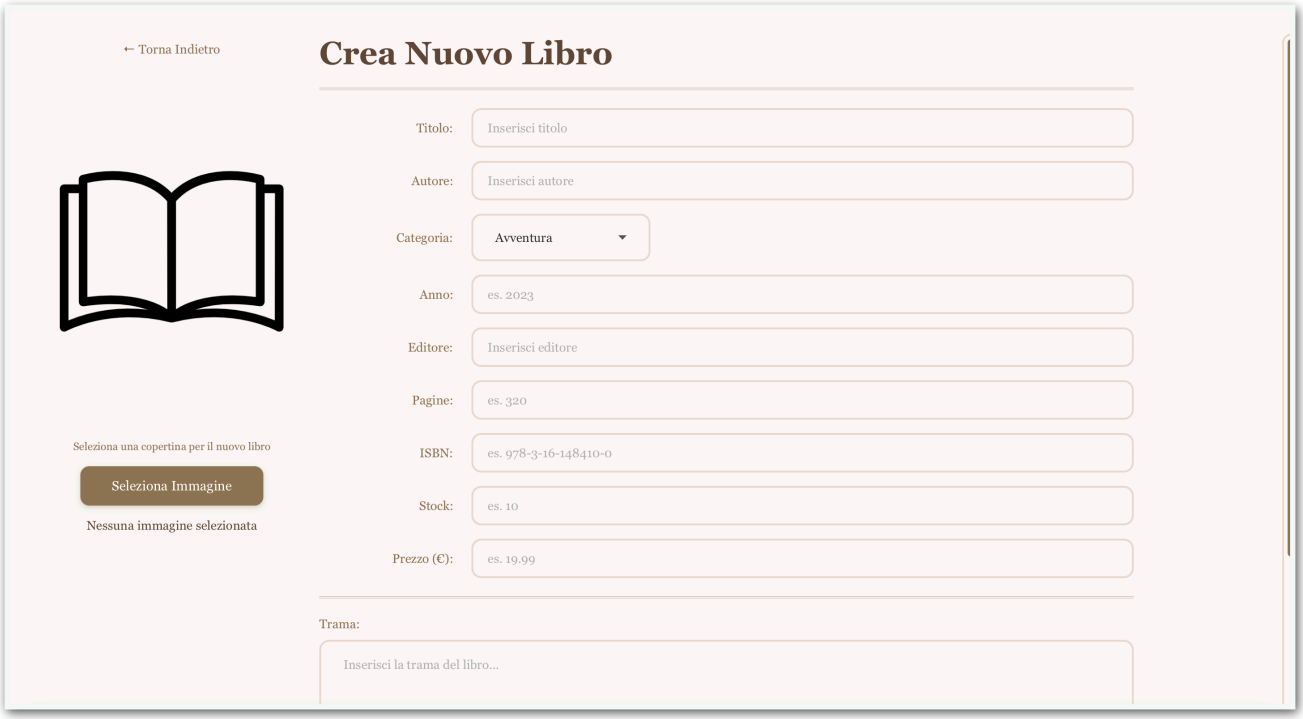
It is used to register the return of loaned books when a user physically returns a borrowed book to the library.

This operation allows the system to correctly update the loan status.

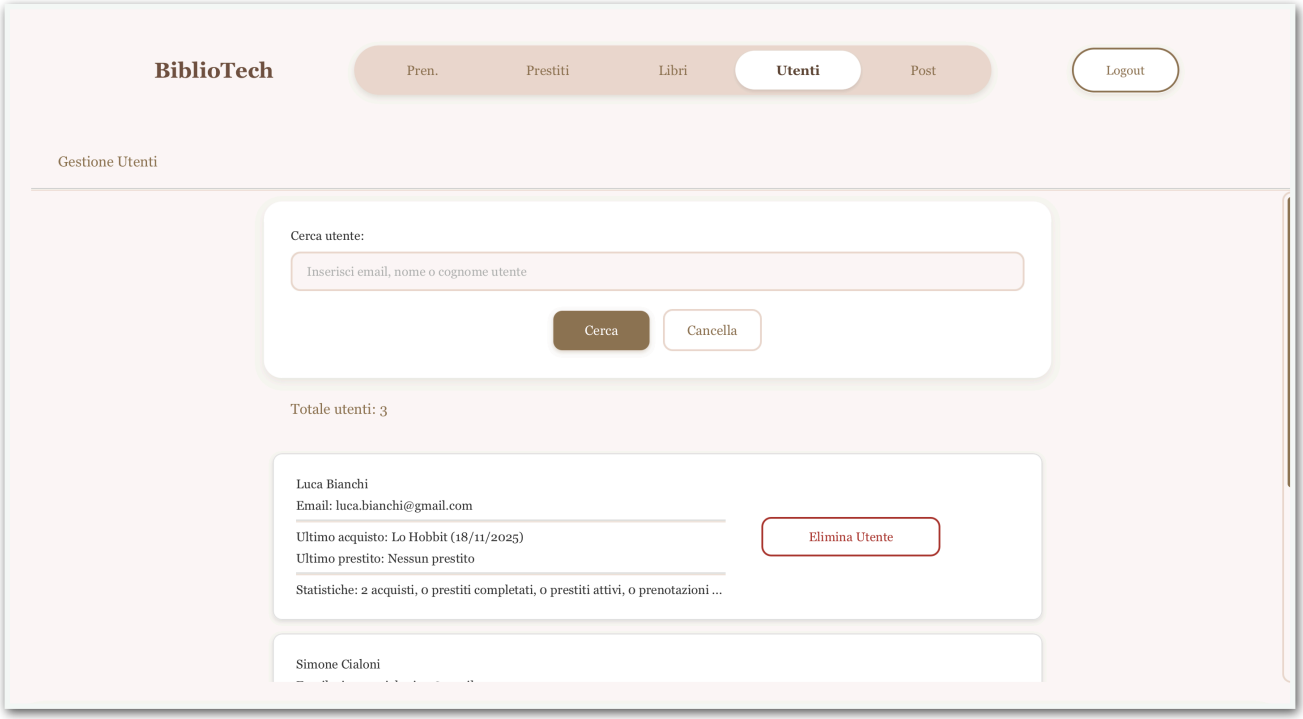


The Manage Books storyboard allows Administrators to search for books and add new books, as well as update or delete existing books in the system.

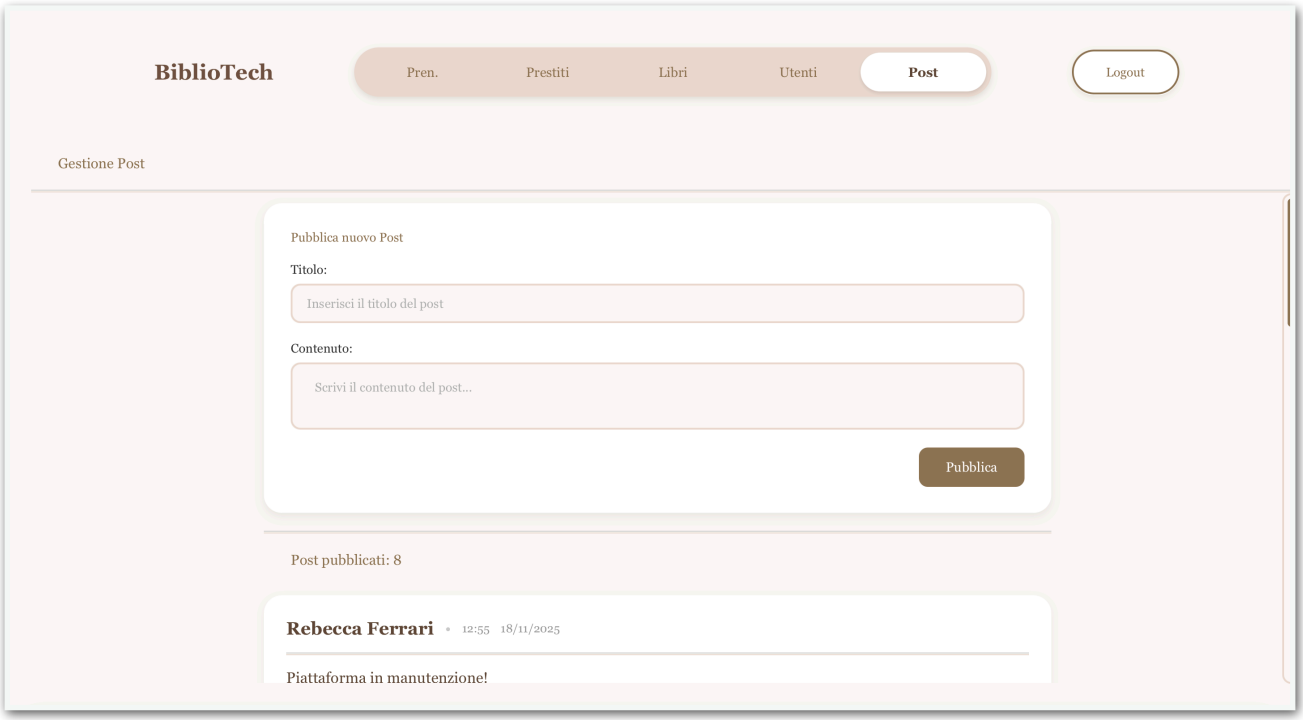




The *Manage Users* storyboard allows Administrators to view user details and manage user accounts. From this interface, administrators can delete user profiles.



The *Post* storyboard allows Administrators to view existing posts and create new ones. This feature enables administrators to publish announcements and updates for system users.

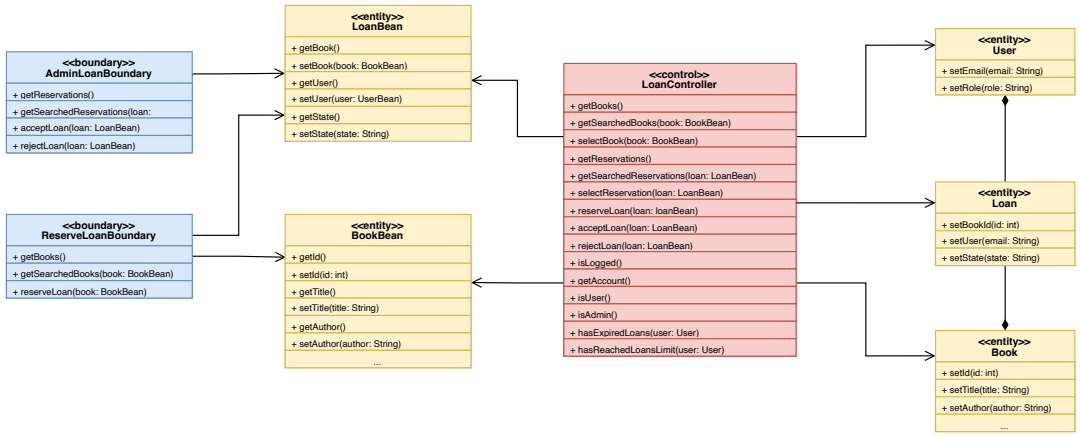


3. DESIGN

3.1 CLASS DIAGRAM

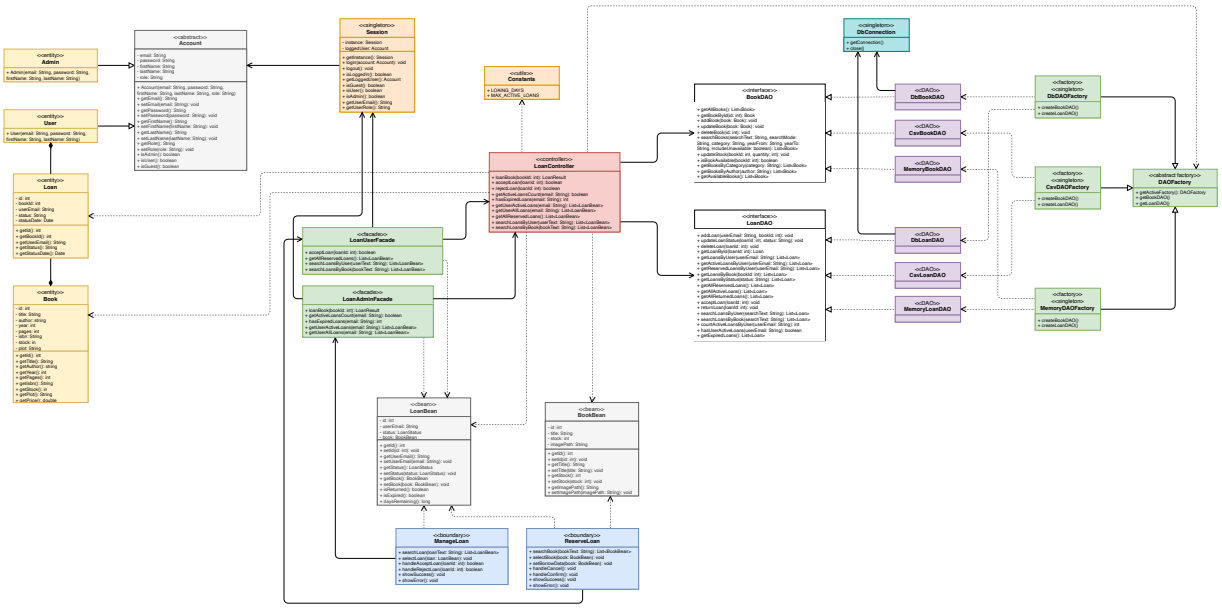
3.1.1. VOPC

Loan Use Case:

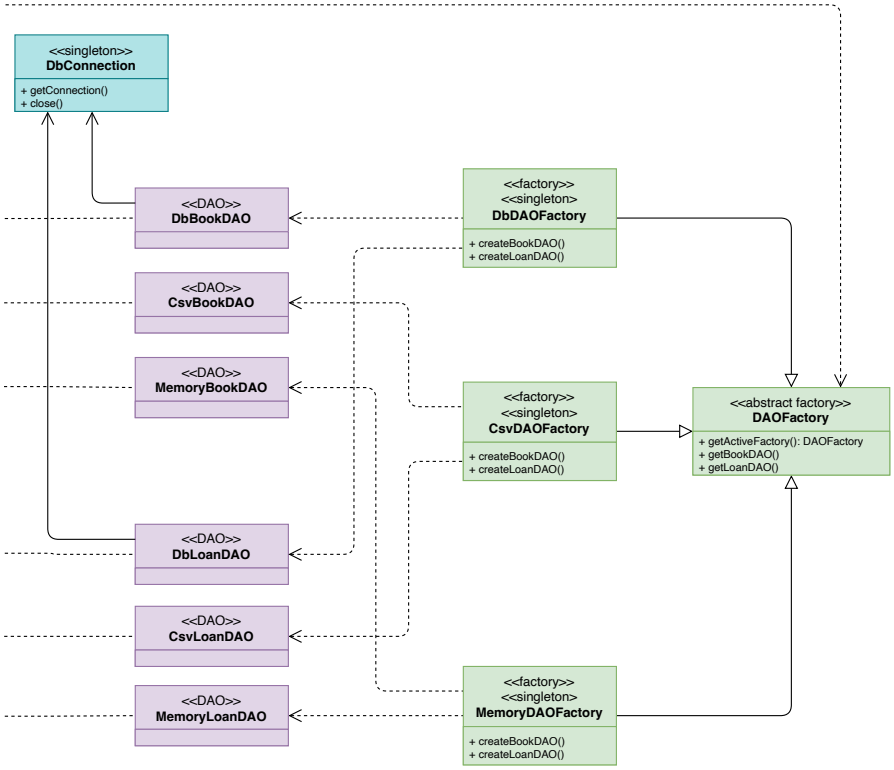


3.1.2. DESIGN-LEVEL DIAGRAM

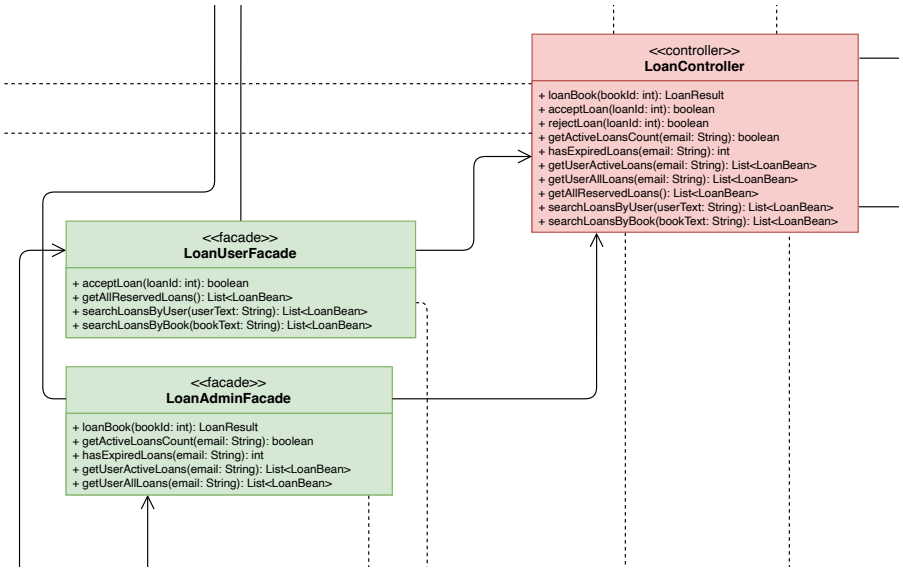
Complete Diagram:



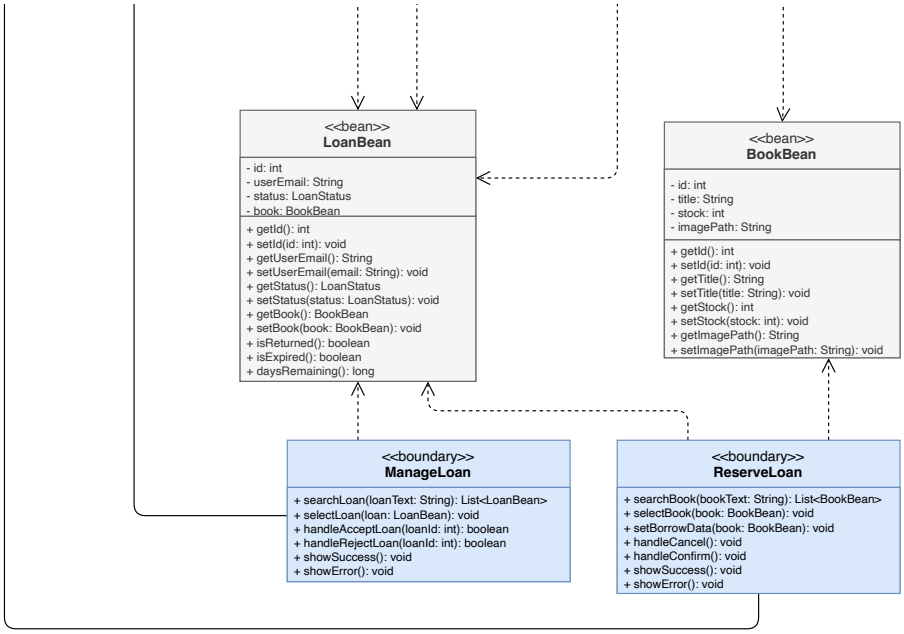
DAO part:



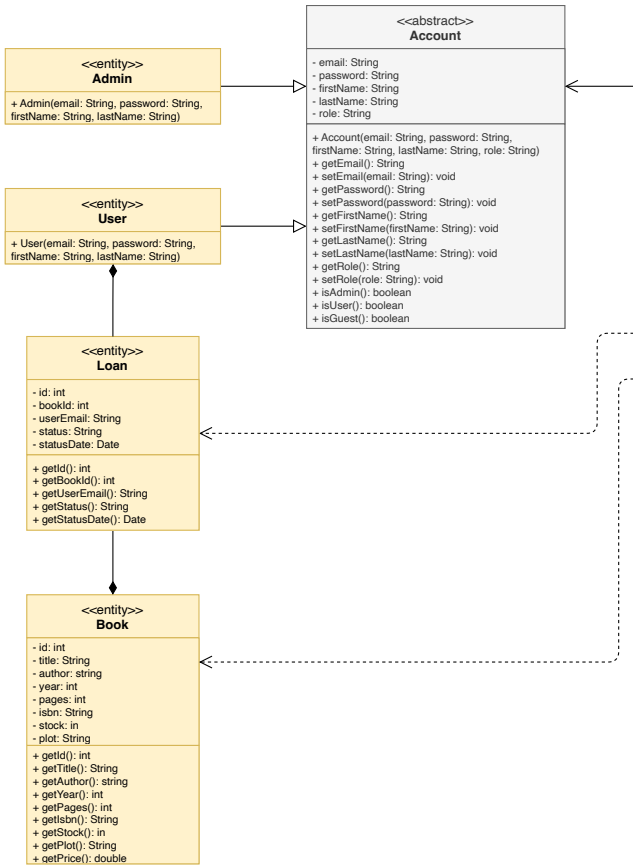
Controller part:



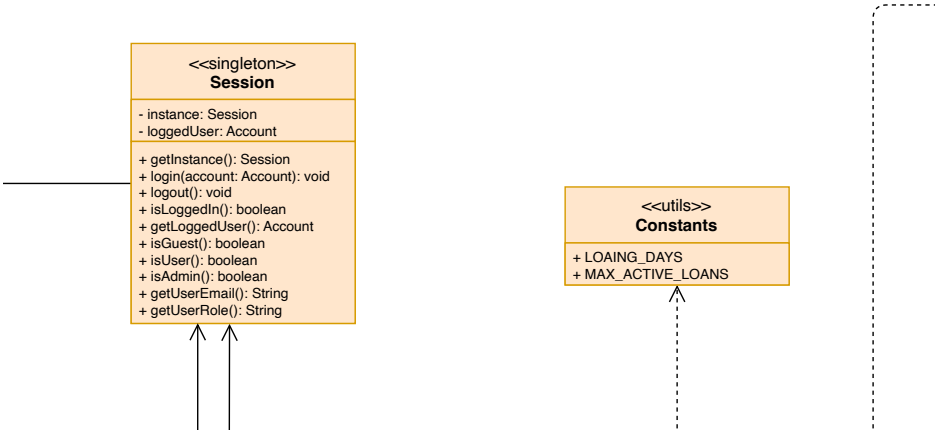
Boundary part:



Model part:



Last part:



3.2. DESIGN PATTERNS

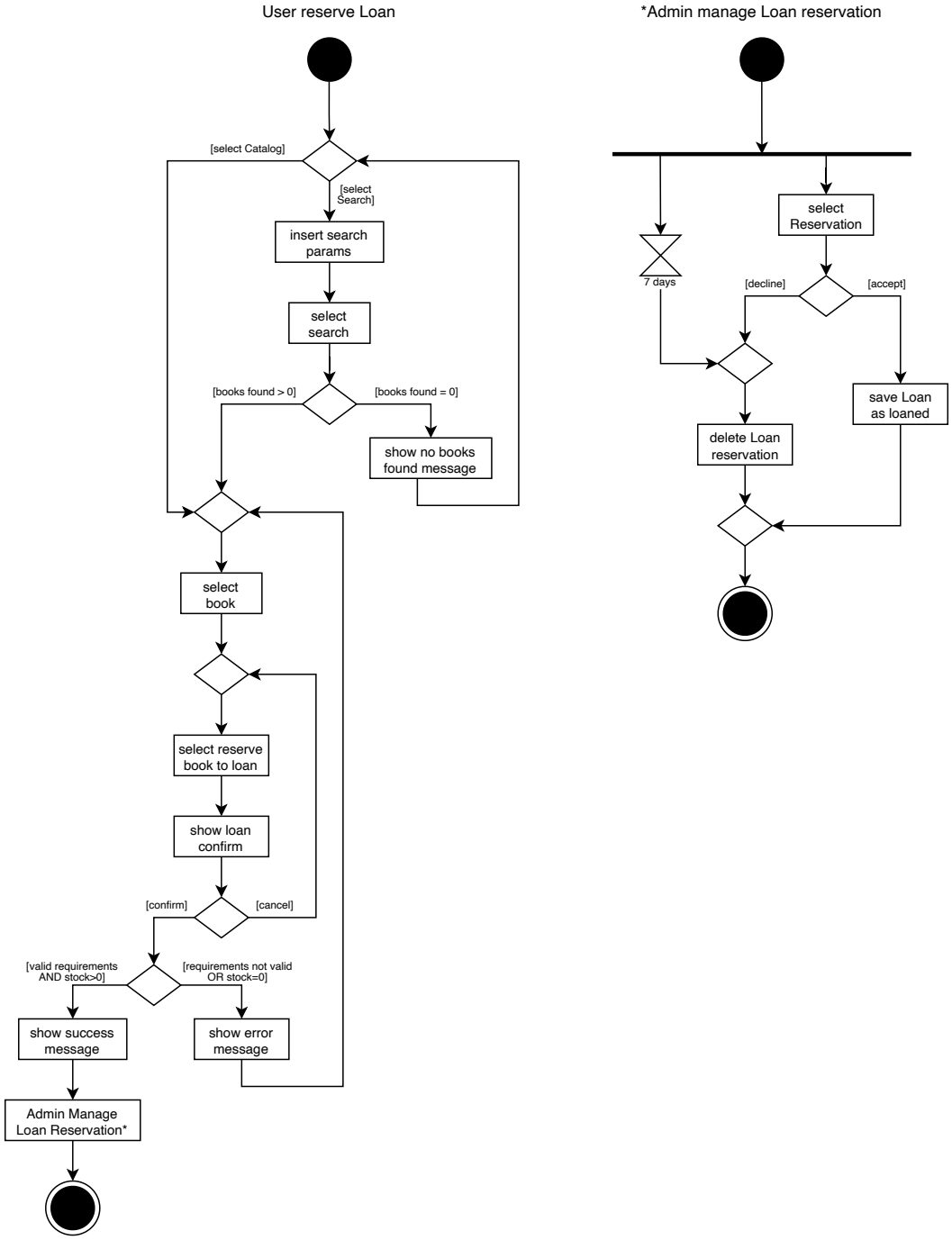
Within the library management system, several architectural design patterns have been adopted to build a solid, maintainable, and secure structure.

The *Facade* pattern is used to clearly separate the responsibilities of regular users and administrators. Instead of exposing a single access point for all loan-related operations in Loan Controller, the system provides one facade for each role, each offering only the functionalities that are appropriate for that user type. This approach prevents regular users from accessing administrative operations and simplifies the interaction with the system by presenting a cleaner and more focused interface. At the same time, the internal business logic remains centralized and hidden behind the facades, which also perform basic access checks based on the user's authentication state and role.

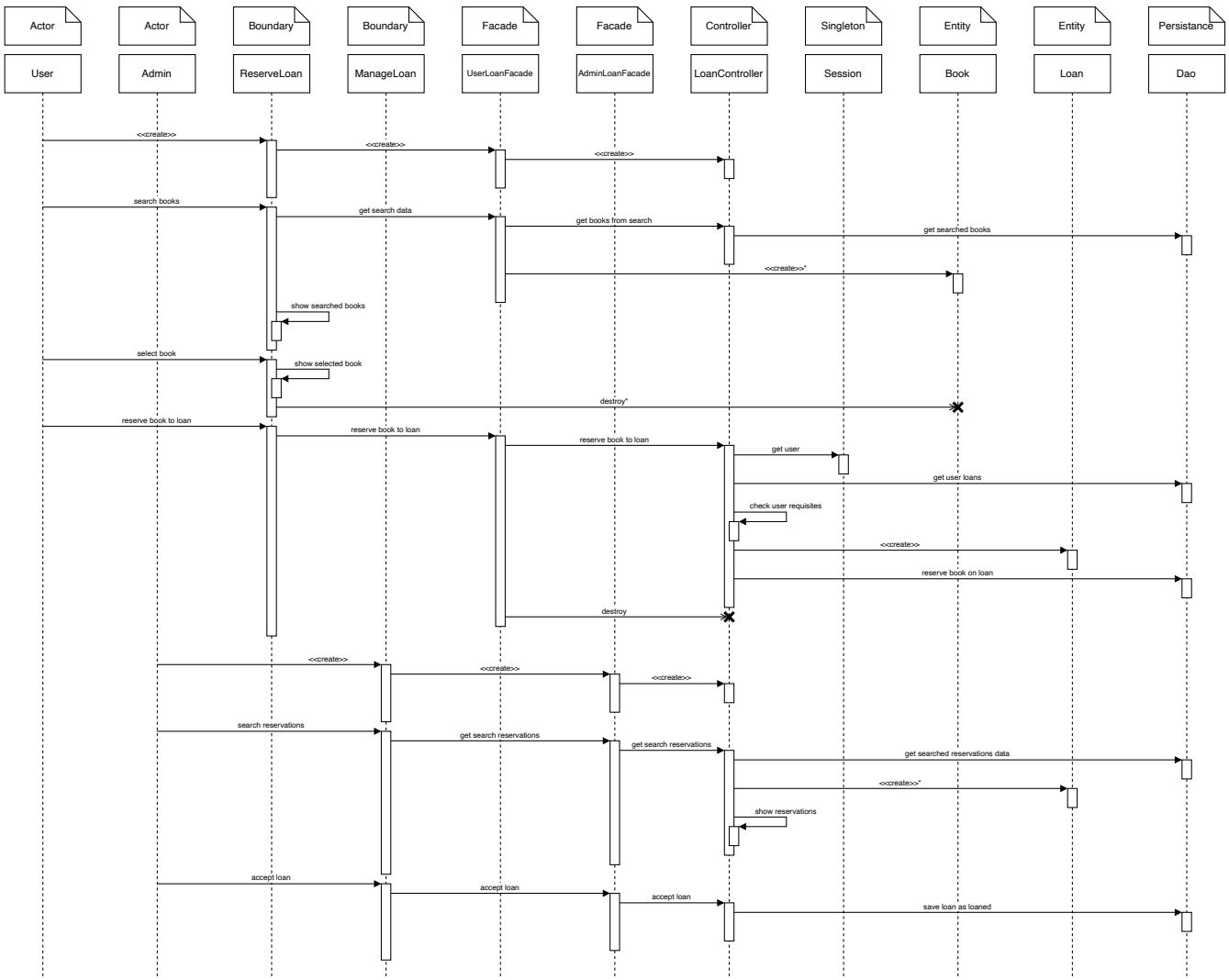
The *Singleton* pattern is applied in parts of the system where having a single shared instance is essential. In particular, session management relies on a unique instance to ensure that the authentication state of the current user is consistent across the entire application. This allows all components to access the same session information without the risk of inconsistencies. A similar approach is used for components responsible for DAO configuration, ensuring uniform interaction with persistent storage. As a result, the overall design is simplified, and coupling between components is reduced.

Together, these patterns contribute to a clear and flexible architecture that improves separation of concerns, supports role-based access, and allows the system to evolve over time with minimal impact on existing components.

3.3. ACTIVITY DIAGRAM

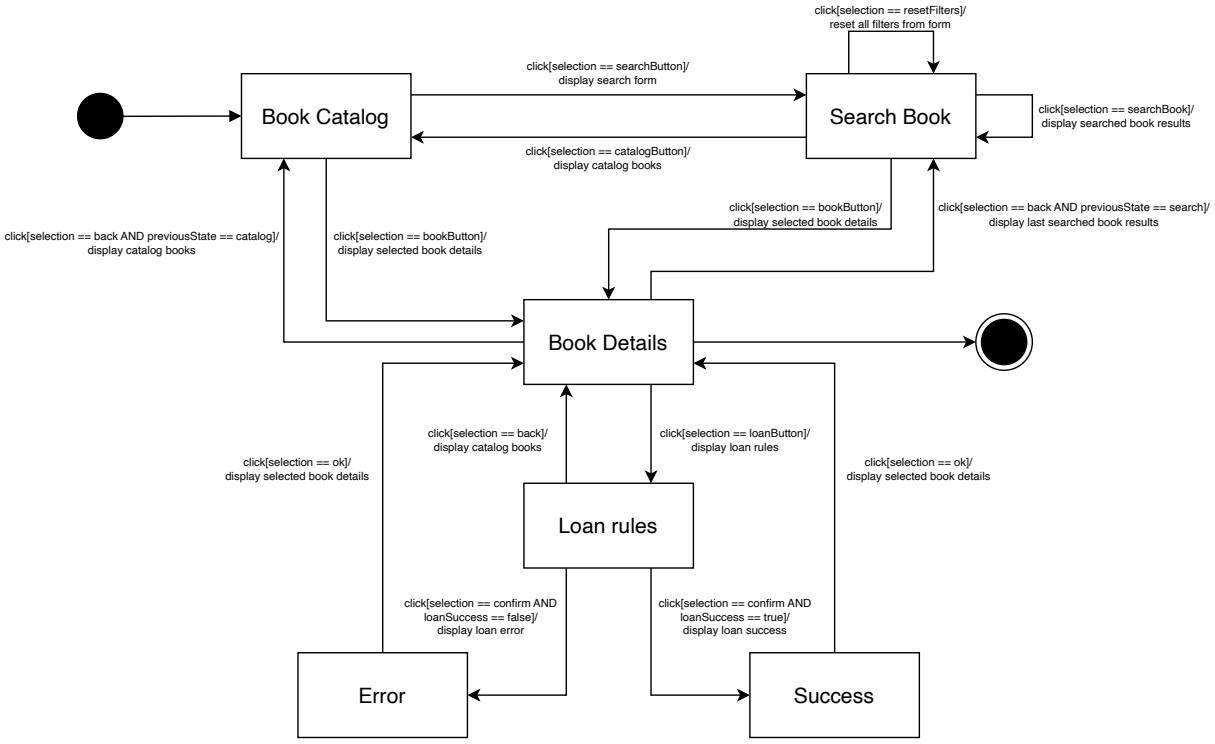


3.4. SEQUENCE DIAGRAM



* le azioni con asterisco vengono iterate N volte in base al numero di risultati ottenuti

3.5. STATE DIAGRAM



4. TESTING

In the testing phase, seven test classes were implemented to verify the correct behavior of the main components of the application.

The LoanBeanTest focuses on the LoanBean class, checking the correct functioning of constructors, setters and getters, as well as validating the loan status through the isReturned() and isExpired() methods.

The SimpleLoanControllerTest verifies elements related to the loan controller, including the correctness of the LoanResult enumeration values and the validity of system constants.

The WishlistEmailObserverTest is dedicated to the email notification mechanism and ensures that users are correctly notified when a book present in their wishlist becomes available.

Similarly, the WishlistObserverTest validates the correct implementation of the Observer pattern by checking that observers are properly notified of relevant events.

The AdminTest class verifies the behavior of the Admin entity by testing constructors, getters, and the logic used to identify administrator users.

The BookTest class focuses on the Book model, testing the Builder pattern with both mandatory and optional fields, validating setters and getters, checking the correctness of equals and hashCode implementations, and ensuring that required fields are properly validated.

Finally, the UserTest class verifies the correct creation of user instances, the behavior of accessor methods, and confirms that standard users are not identified as administrators.

Overall, these tests cover data models, business logic such as loan expiration and observer mechanisms, and integration aspects like email notifications, using a combination of unit tests and tests with mocked dependencies.

The test classes are analyzed can be found at the following link: GitHub Test Repository.

5. CODE

The source code of the project is available on GitHub at the following link: GitHub Repository.

The code quality analysis is available on SonarCloud at the following link: SonarCloud Analysis.

6. VIDEO

A video demonstration showing how the application works is available on YouTube at the following link: YouTube Video.